

3.5 a 题目要求判断一个周期为 5 的信号类型。

分析：由题目可知，由于这个信号是周期信号，因此其傅里叶级数也是周期的，且 $a(n) = a(n+5)$ ，又因为 $a(2)=a^*(-2)$ ， $a(4) = a^*(-4)$ ， $a_0=1$ ，容易知道 $a(k)=a^*(-k)$ ，由傅里叶级数性质易得 $x[n]$ 为纯实信号。

结论： $x[n]$ 是纯实信号。

3.5 b 题目要求

题目要求利用离散时间傅里叶级数系数确定 a_0 到 a_4 的值，同时构造一个包含这些值的向量 a 。

分析：因为周期为 5，根据离散时间傅里叶级数的性质， $a(n) = a(n+5)$

因此有：

$$a(0) = 1$$

$$a(1) = a(-4) = [2e^{j(\pi/3)}]^* = 2e^{j(-\pi/3)}$$

$$a(2)=a(-3)=(e^{j(\pi/4)})^*=e^{j(\pi/4)}$$

$$a(3) = a(-2) = (e^{j(\pi/4)})^* = e^{j(-\pi/4)}$$

$$a_4 = 2e^{j(\pi/3)}$$

代码如图所示：

```
an = 0:4;
a = [1, 2.*exp(1i.*(-pi/3)), exp(1i.*(pi/4)), exp(1i.*(-pi/4)), 2.*exp(1i.*(pi/3))];
```

结论：构造向量如图所示。

3.5c 题目要求利用上述级数向量 a 和合成公式，定义一个包含离散时间信号的 $x[n]$ 一个周期的 ($0 \leq n \leq 4$) 新向量 x ，并作出相关图像，验证预测是否正确。

分析：使用 for 循环构建向量，并作出 $x[n]$ 的信号。

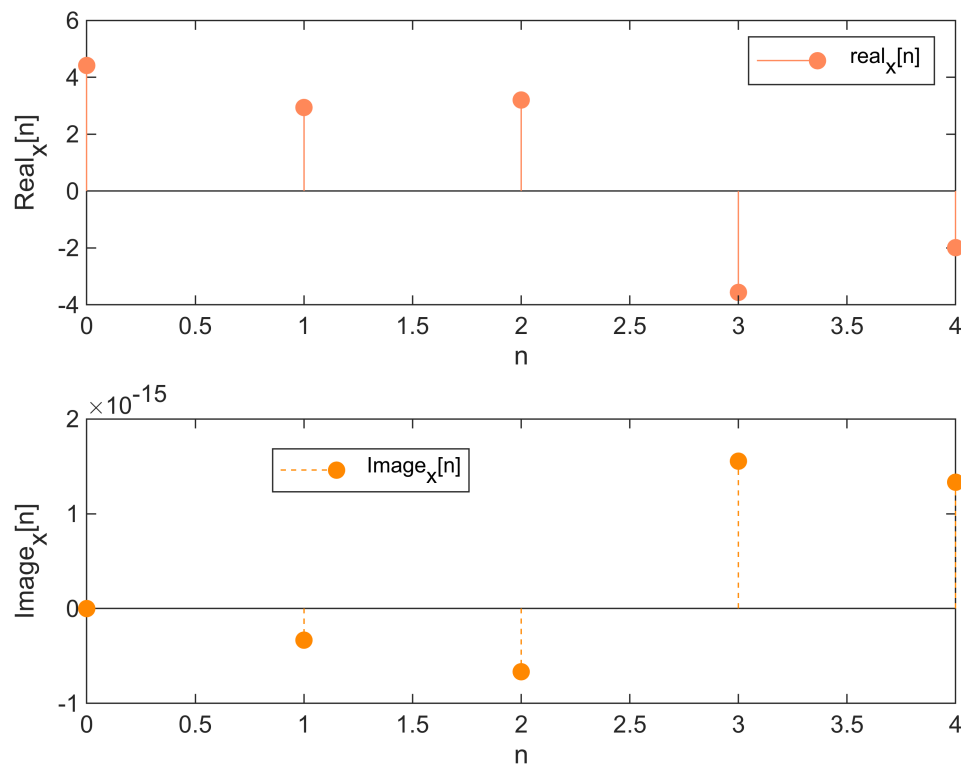
代码如下：

```
xn = 0:1:4;
x=[0,0,0,0,0];
for p = 1:5
    for j = 1:5
        x(p)=x(p)+a(j).*exp(1i.*(j-1).*(2.*pi/5).*(p-1));
    end
end
figure;
subplot(2,1,1);
stem(xn,real(x),"filled","LineStyle","-", "Color",[255,136,89]/255)
xlabel('n')
ylabel("Real_x[n]")
legend('real_x[n]')
subplot(2,1,2);
```

```

stem(xn,imag(x),'filled',"LineStyle","--","Color",[255,136,0]/255)
xlabel('n')
ylabel("Image_x[n]")
legend("Image_x[n]","Position", [0.27897,0.35601,0.20898,0.0625])

```



结论：根据绘制的图像可以判断 $x[n]$ 是纯实信号的判断。存在 $\text{Image}_x[n]$ 的原因是有系统误差，可忽略不计。

3.5 d 题目要求构造出向量 x_1, x_2, x_3 来表示题目中 3 个周期信号 $x_1[n], x_2[n], x_3[n]$ 并作图。（ $0 \leq n \leq 63$ ）。

分析：按照要求构造向量，代码如下：

```

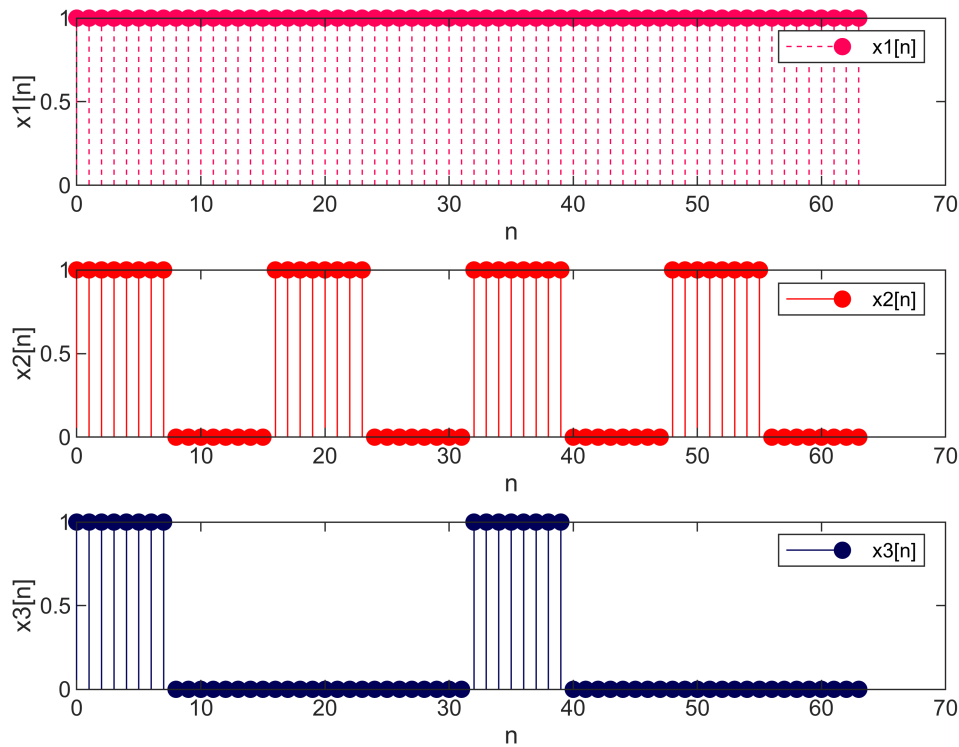
xn1 = 0:63;
x1 = [ones(1,8)];
x2 = [ones(1,8),zeros(1,8)];
x3 = [ones(1,8),zeros(1,24)];
figure;
subplot(3,1,1)
stem(xn1,[x1,x1,x1,x1,x1,x1,x1,x1], 'filled', "LineStyle", "--", "Color", [255,0,89]/255)
xlabel('n')
ylabel('x1[n]')
legend('x1[n]')
subplot(3,1,2)
stem(xn1,[x2,x2,x2,x2], 'filled', "LineStyle", "--", "Color", [255,0,0]/255);
xlabel('n')

```

```

ylabel('x2[n]')
legend('x2[n]')
subplot(3,1,3);
stem(xn1,[x3,x3],'filled',"LineStyle","-","Color",[0,0,89]/255);
xlabel('n')
ylabel('x3[n]')
legend('x3[n]')

```



结论：图像如图所示。

3.5e,题目要求使用 fft 函数求出 a_1, a_2, a_3 分别为信号 $x_1[n], x_2[n], x_3[n]$ 的离散时间傅里叶级数系数。并做出图像。同时根据(d)和 Eq.(3.2),进行 $a_1(1), a_2(1), a_3(1)$ 直流分量的预测, 并判断预测正确性。

分析：根据 d 问可得 $a_1(1)=1, a_2(1) = 1/2, a_3(1)=1/4$;

```

an1 = 0:7;
a1 = (1/8).*fft(x1);
an2 = 0:15;
a2 = (1/16).*fft(x2);
an3 = 0:31;
a3 = (1/32).*fft(x3);
figure;

subplot(3,1,1);
stem(an1,abs(a1),'filled',"LineStyle","-","Color",[45,123,89]/255)

```

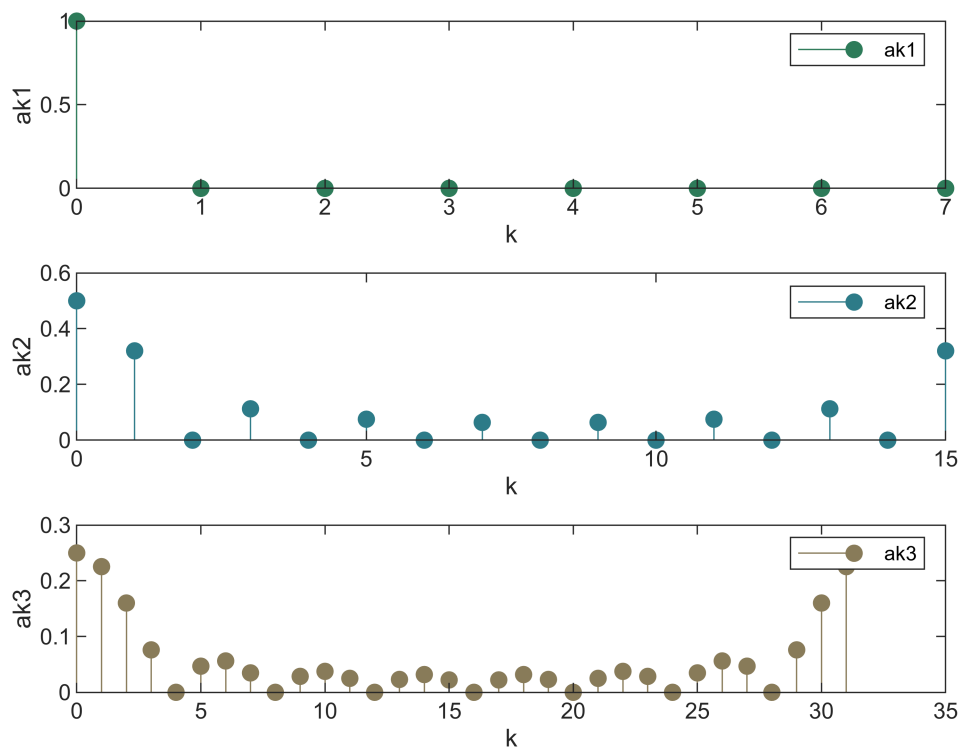
```

xlabel('k')
ylabel('ak1')
legend('ak1')

subplot(3,1,2);
stem(an2,abs(a2),'filled',"LineStyle","-", "Color",[45,123,136]/255)
xlabel('k')
ylabel('ak2')
legend('ak2')

subplot(3,1,3);
stem(an3,abs(a3),'filled',"LineStyle","-", "Color",[136,123,89]/255)
xlabel('k')
ylabel('ak3')
legend('ak3')

```



结论：如图所示，根据图像，预测的 $a_1(1), a_2(1), a_3(1)$ 的值与图像相匹配。

3.5 f, 根据题目，利用 (e) 问中的 a_3 , 构造向量 $x_3_2, x_3_8, x_3, x_3_all$ 分别与题目中的 $x_3_2[n], x_3_8[n], x_3_12[n], x_3_all[n]$, 相对应，并作出图像。

分析：作出 $x_3_2, x_3_8, x_3, x_3_all$ 图像：

```

xn3=0:31;

```

```

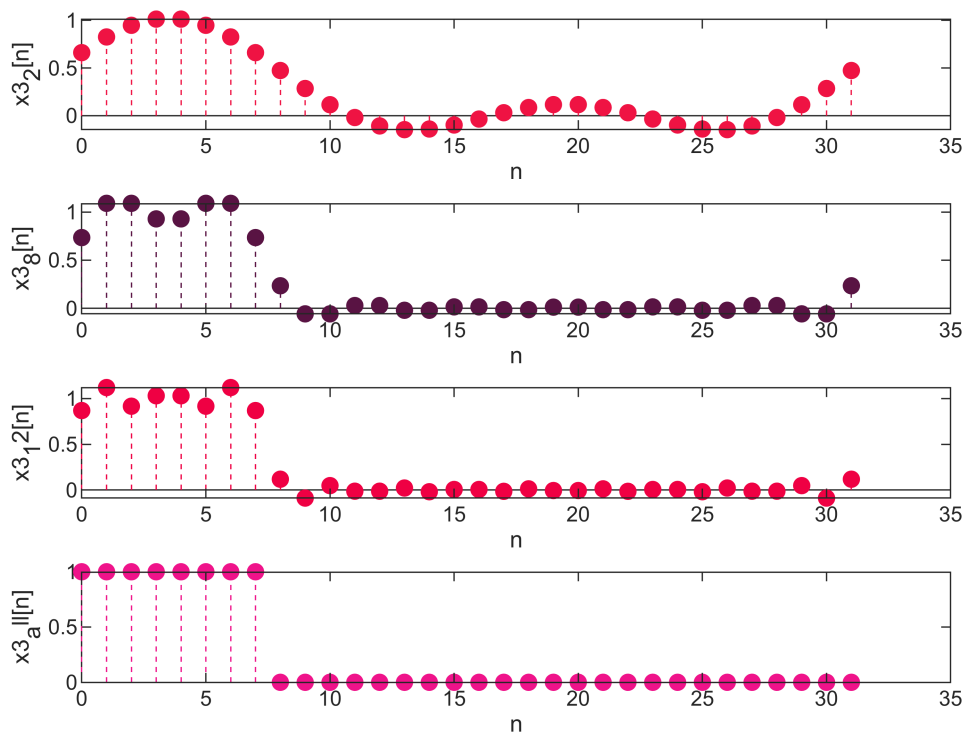
x3_2=zeros(1,32);
a3_2 = [a3(31),a3(32),a3(1),a3(2),a3(3)];
for p = 1:32
    for j = 1:5
        x3_2(p) = x3_2(p)+a3_2(j).*exp(1i.*(j-3).*(2.*pi/32).*(p-1));
    end
end
x3_8=zeros(1,32);
a3_8 = [a3(25),a3(26),a3(27),a3(28),a3(29),a3(30),a3(31),a3(32),a3(1),a3(2),a3(3),a3(4),a3(5),a3(6),a3(7),a3(8),a3(9),a3(10),a3(11),a3(12),a3(13),a3(14),a3(15),a3(16),a3(17),a3(18),a3(19),a3(20),a3(21),a3(22),a3(23),a3(24)];
for p = 1:32
    for j = 1:17
        x3_8(p) = x3_8(p)+a3_8(j).*exp(1i.*(j-9).*(2.*pi/32).*(p-1));
    end
end
x3_12=zeros(1,32);
a3_12=[a3(21) a3(22) a3(23) a3(24) a3(25) a3(26) a3(27) a3(28) a3(29) a3(30) a3(31) a3(32) a3(1) a3(2) a3(3) a3(4) a3(5) a3(6) a3(7) a3(8) a3(9) a3(10) a3(11) a3(12) a3(13) a3(14) a3(15) a3(16) a3(17) a3(18) a3(19) a3(20)];
for p = 1:32
    for j = 1:25
        x3_12(p) = x3_12(p)+a3_12(j).*exp(1i.*(j-13).*(2.*pi/32).*(p-1));
    end
end
x3_all=zeros(1,32);
a3_all=zeros(1,32);
a3_all=[a3(18),a3(19),a3(20),a3(21),a3(22),a3(23),a3(24),a3(25),a3(26),a3(27),a3(28),a3(29),a3(30),a3(31),a3(32),a3(1),a3(2),a3(3),a3(4),a3(5),a3(6),a3(7),a3(8),a3(9),a3(10),a3(11),a3(12),a3(13),a3(14),a3(15),a3(16),a3(17)];
for p = 1:32
    for j = 1:32
        x3_all(p) = x3_all(p)+a3_all(j).*exp(1i.*(j-16).*(2.*pi/32).*(p-1));
    end
end
figure;
subplot(4,1,1);
stem(xn3,real(x3_2),'filled','LineStyle','--','Color',[239,19,67]/255)
xlabel('n')
ylabel('x3_2[n]')

subplot(4,1,2);
stem(xn3,real(x3_8),'filled','LineStyle','--','Color',[89,19,67]/255)
xlabel('n')
ylabel('x3_8[n]')

subplot(4,1,3);
stem(xn3,real(x3_12),'filled','LineStyle','--','Color',[239,0,67]/255)
xlabel('n')
ylabel('x3_12[n]')

```

```
subplot(4,1,4);
stem(xn3,real(x3_all),'filled','LineStyle','--','Color',[239,19,138]/255)
xlabel('n')
ylabel('x3_all[n]')
```

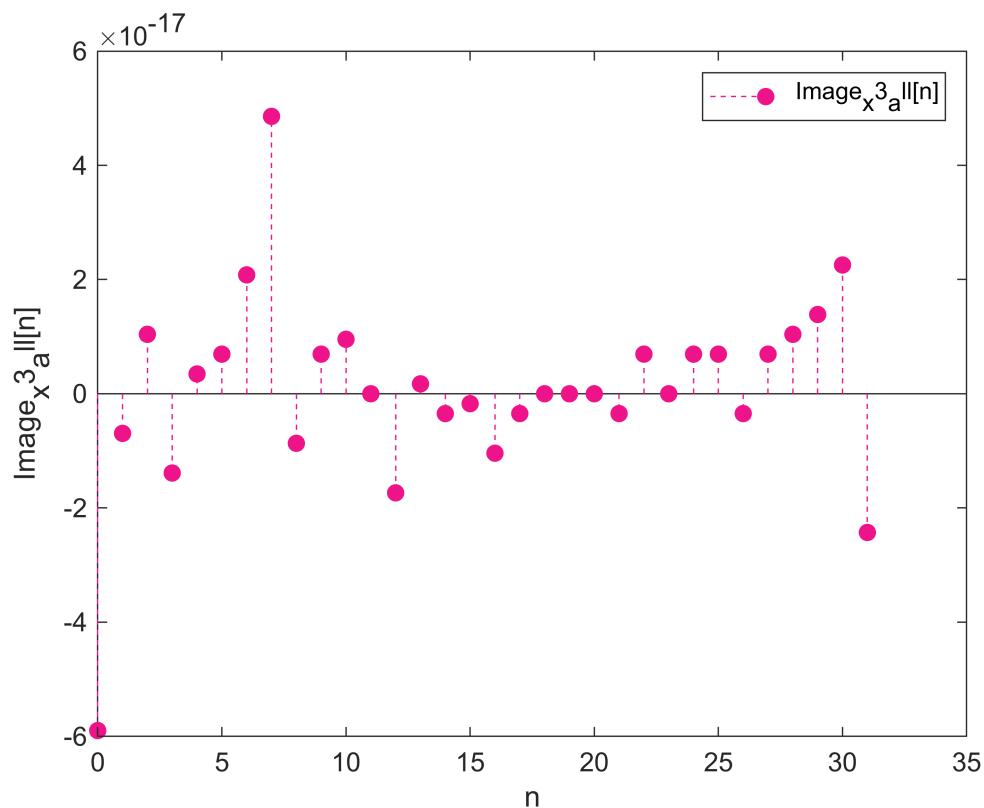


结论：定义向量及做出的图片如上，可以看到，随着一个谐波集中越多的信号进行合成，结果越逼近原信号。

3.5g 题目要求证明 $x3_all[n]$ 是一个实信号。

分析：要证明其是实信号，只需要画出其虚部，观察其虚部数值即可

```
figure;
stem(xn3,imag(x3_all),'filled','LineStyle','--','Color',[239,19,138]/255)
xlabel('n')
ylabel('Image_x3_all[n]')
legend('Image_x3_all[n]')
```



从图像可看出 $x3_all$ 的虚部图像可看出其虚部信号很小，可以忽略不计（由于系统误差才产生虚部信号）

下面给出证明：

$x3[n]$ 是实信号， 根据离散时间傅里叶级数性质可知：离散时间负指数信号具有周期性。

离散时间负指数信号具有周期性，一个谐波信号集内有 $N=32$ 的离散时间负指数信号

同时离散时间傅里叶级数也具有周期性， $N=32$. 因此利用任何一个完整地谐波信号

集与 $N=32$ 个连续的离散时间傅里叶级数都可以合成出原信号，即 $x3_all[n]=x3[n]$

因此 $x3_all[n]$ 一定是一个实信号。

3.5 h 题目要求绘制一系列图像，以表示随着使用一个谐波信号集中越多的信号进行合成，结果越来越逼近原信号 $x3[n]$ 的过程，同时在误差允许范围内，观察 $x3_all[n]=x3[n]$ 。最后观察是否会出现吉布斯现象。

分析：代码如下：

```
figure;
x3_4=zeros(1,32)];
a3_4=[a3(29) a3(30) a3(31) a3(32) a3(1) a3(2) a3(3) a3(4) a3(5)];
for p=1:32
    for j=1:9
        x3_4(p)=x3_4(p)+a3_4(j).*exp(1i.*(j-5).*(2.*pi/32).*(p-1));
```

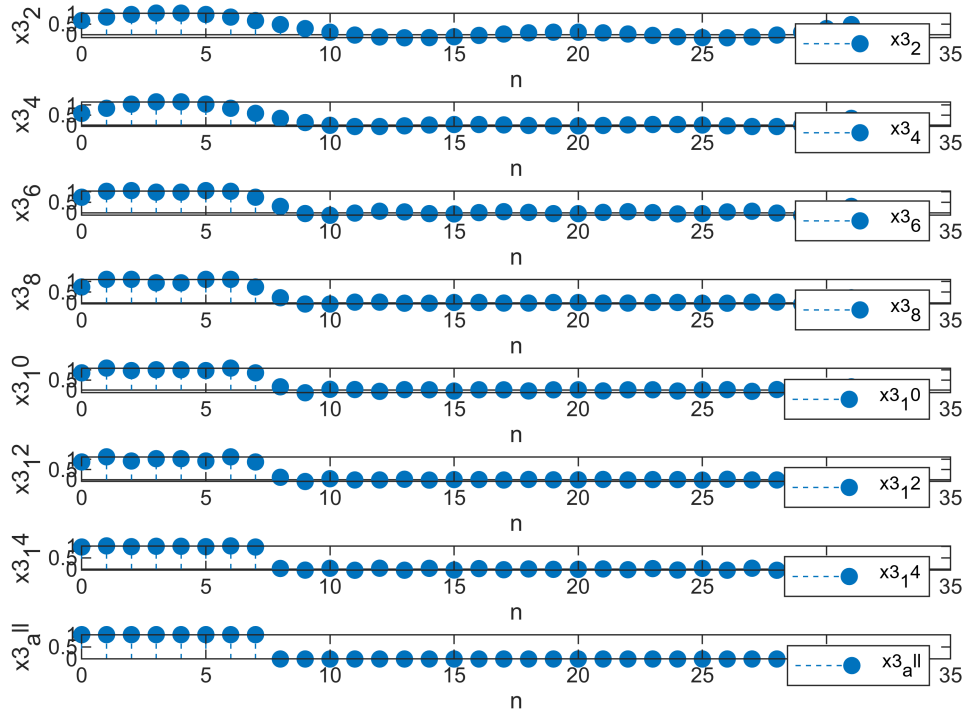
```

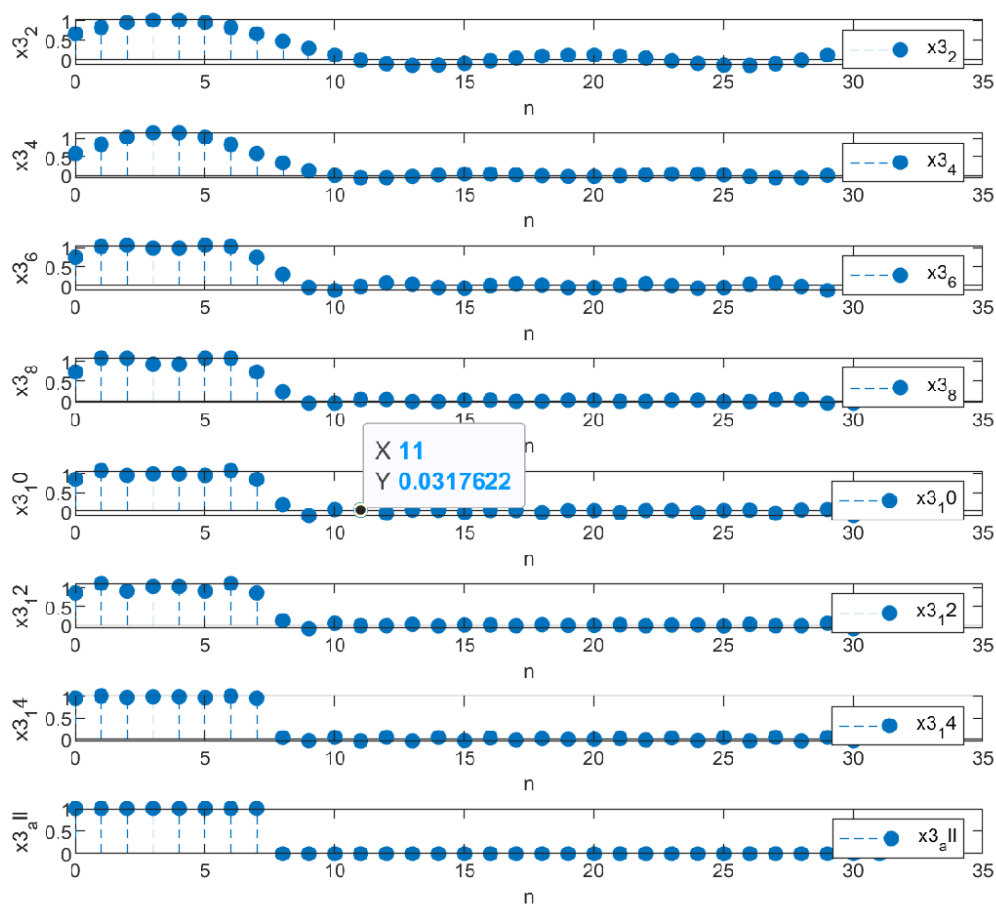
end
end
x3_6=zeros(1,32);
a3_6=[a3(27) a3(28) a3(29) a3(30) a3(31) a3(32) a3(1) a3(2) a3(3) a3(4) a3(5) a3(6) a3(7)];
for p=1:32
    for j=1:13
        x3_6(p)=x3_6(p)+a3_6(j).*exp(1i.*(j-7).*(2.*pi/32).*(p-1));
    end
end
x3_10=zeros(1,32);
a3_10=[a3(23) a3(24) a3(25) a3(26) a3(27) a3(28) a3(29) a3(30) a3(31) a3(32) a3(1) a3(2) a3(3)
a3(4) a3(5) a3(6) a3(7) a3(8) a3(9) a3(10) a3(11) a3(12) a3(13) a3(14) a3(15) a3(16) a3(17) a3(18)];
for p=1:32
    for j=1:21
        x3_10(p)=x3_10(p)+a3_10(j).*exp(1i.*(j-11).*(2.*pi/32).*(p-1));
    end
end

x3_14=zeros(1,32);
a3_14=[a3(19) a3(20) a3(21) a3(22) a3(23) a3(24) a3(25) a3(26) a3(27) a3(28) a3(29) a3(30) a3(31) a3(32)
a3(1) a3(2) a3(3) a3(4) a3(5) a3(6) a3(7) a3(8) a3(9) a3(10) a3(11) a3(12) a3(13) a3(14) a3(15) a3(16) a3(17) a3(18)];
for p=1:32
    for j=1:29
        x3_14(p)=x3_14(p)+a3_14(j).*exp(1i.*(j-15).*(2.*pi/32).*(p-1));
    end
end

x3n = [x3_2;x3_4;x3_6;x3_8;x3_10;x3_12;x3_14;x3_all];
x3n1= ["x3_2", "x3_4", "x3_6", "x3_8", "x3_10", "x3_12", "x3_14", "x3_all"];
for i = 1:8
    subplot(8,1,i)
    stem(xn3,real(x3n(i,:)), 'filled', 'LineStyle', '--');
    xlabel('n')
    ylabel(x3n1(i))
    legend(x3n1(i))
end

```



结论：由图可知随着使用一个谐波信号集中越来越多的信号进行合成，结果越逼近原信号 $x_3[n]$ 。同时不会出现吉布斯现象。

3.5 i 题目要求自己写出 DTFS 函数算法，并检验。

分析：按照题目要求，写出对应函数后代入检验。

```
dtfs([1,2,3,4],0)
```

```
ans = 1x4 complex
2.5000 + 0.0000i -0.5000 + 0.5000i -0.5000 - 0.0000i -0.5000 - 0.5000i
```

```
dtfs([1,2,3,4],1)
```

```
ans = 1x4 complex
2.5000 + 0.0000i 0.5000 + 0.5000i 0.5000 - 0.0000i 0.5000 - 0.5000i
```

```
dtfs([1,2,3,4],-1)
```

```
ans = 1x4 complex  
2.5000 + 0.0000i -0.5000 - 0.5000i 0.5000 + 0.0000i -0.5000 + 0.5000i
```

```
dtfs([2,3,4,1],0)
```

```
ans = 1x4 complex  
2.5000 + 0.0000i -0.5000 - 0.5000i 0.5000 + 0.0000i -0.5000 + 0.5000i
```

```
dtfs([ones(1,4) zeros(1,4)],0)
```

```
ans = 1x8 complex  
0.5000 + 0.0000i 0.1250 - 0.3018i -0.0000 - 0.0000i 0.1250 - 0.0518i ...
```

```
dtfs([ones(1,4) zeros(1,4)],2)
```

```
ans = 1x8 complex  
0.5000 + 0.0000i -0.3018 - 0.1250i 0.0000 + 0.0000i 0.0518 + 0.1250i ...
```

结论：测试结果与题目一致。

3.8a 定义 a_1, b_1 是系统 1 的 filter 和 freqz 函数的系数, a_2, b_2 是系统 2 的 filter 和 freqz 函数的系数, 表示 a_1, a_2, b_1, b_2

分析：根据 b, a 的定义来写

```
a1=[1, -0.8]
```

```
a1 = 1x2  
1.0000 -0.8000
```

```
b1=[1]
```

```
b1 = 1
```

```
a2=[1, 0.8]
```

```
a2 = 1x2  
1.0000 0.8000
```

```
b2=[1]
```

```
b2 = 1
```

结论： $a_1=[1 \ -0.8]$; $b_1=[1 \ 0]$; $a_2=[1 \ 0.8]$; $b_2=[1 \ 0]$;

3.8b 用 freqz 函数求系统 1 和系统 2 的频率响应

分析：用 freqz 方法,频率在奇数 π 附近是高频, 在偶数 π 附近是低频

```
figure
```

计算的范围为 $0-2\pi$

```
a=linspace(2,2*pi,1024)
```

```
a = 1×1024  
    2.0000    2.0042    2.0084    2.0126    2.0167    2.0209    2.0251    2.0293 ...
```

用 freqz 函数计算频率响应 H1,H2

```
[H1,a]=freqz(b1,a1,1024,'whole')
```

```
H1 = 1024×1 complex  
    5.0000 + 0.0000i  
    4.9966 - 0.1226i  
    4.9865 - 0.2447i  
    4.9697 - 0.3657i  
    4.9464 - 0.4850i  
    4.9169 - 0.6022i  
    4.8812 - 0.7167i  
    4.8399 - 0.8282i  
    4.7931 - 0.9362i  
    4.7414 - 1.0405i  
      ⋮  
      ⋮  
a = 1024×1  
      0  
    0.0061  
    0.0123  
    0.0184  
    0.0245  
    0.0307  
    0.0368  
    0.0430  
    0.0491  
    0.0552  
      ⋮  
      ⋮
```

```
[H2,a]=freqz(b2,a2,1024,'whole')
```

```
H2 = 1024×1 complex  
    0.5556 + 0.0000i  
    0.5556 + 0.0015i  
    0.5556 + 0.0030i  
    0.5556 + 0.0045i  
    0.5556 + 0.0061i  
    0.5556 + 0.0076i  
    0.5556 + 0.0091i  
    0.5556 + 0.0106i  
    0.5556 + 0.0121i  
    0.5556 + 0.0136i  
      ⋮  
      ⋮  
a = 1024×1
```

```

0
0.0061
0.0123
0.0184
0.0245
0.0307
0.0368
0.0430
0.0491
0.0552
⋮

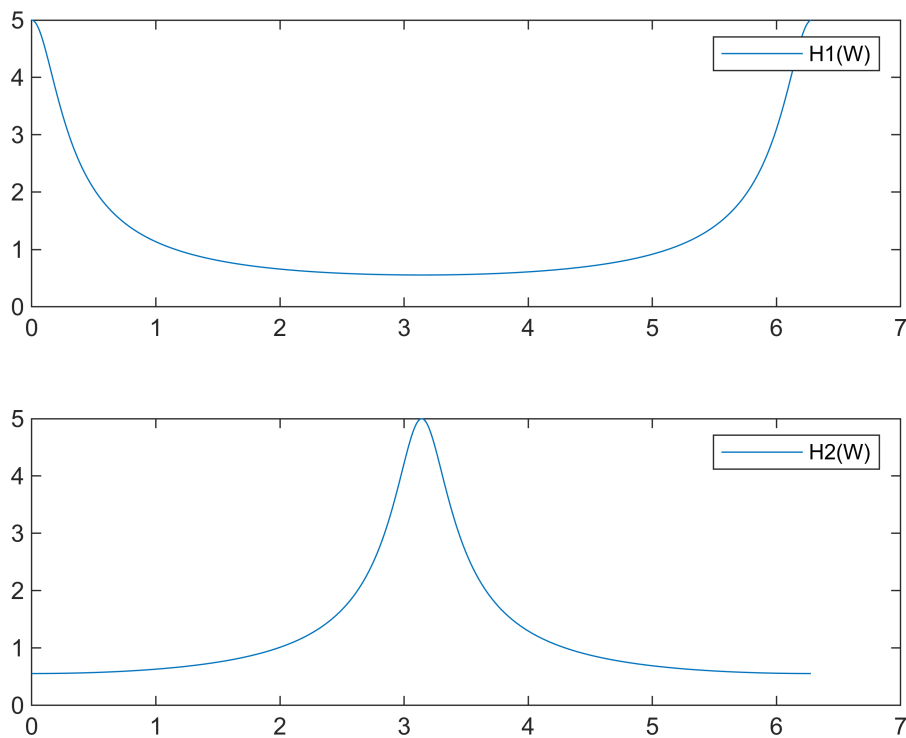
```

画出 H1,H2 的图像

```

subplot(2,1,1),plot(a,abs(H1))
legend('H1(W)')
subplot(2,1,2),plot(a,abs(H2))
legend('H2(W)')

```



由图可知，系统 1 是低通滤波，系统 2 是高通滤波

3.8c 定义 a_x 是 $x[n]$ 的傅里叶级数，画出 $w_k=(2\pi/20)k$ 时的 a_x 的图像，判断与 b 题的图像相比，哪一部分频率被放大，哪一部分被缩小

```
figure
```

由题可表示 a_k

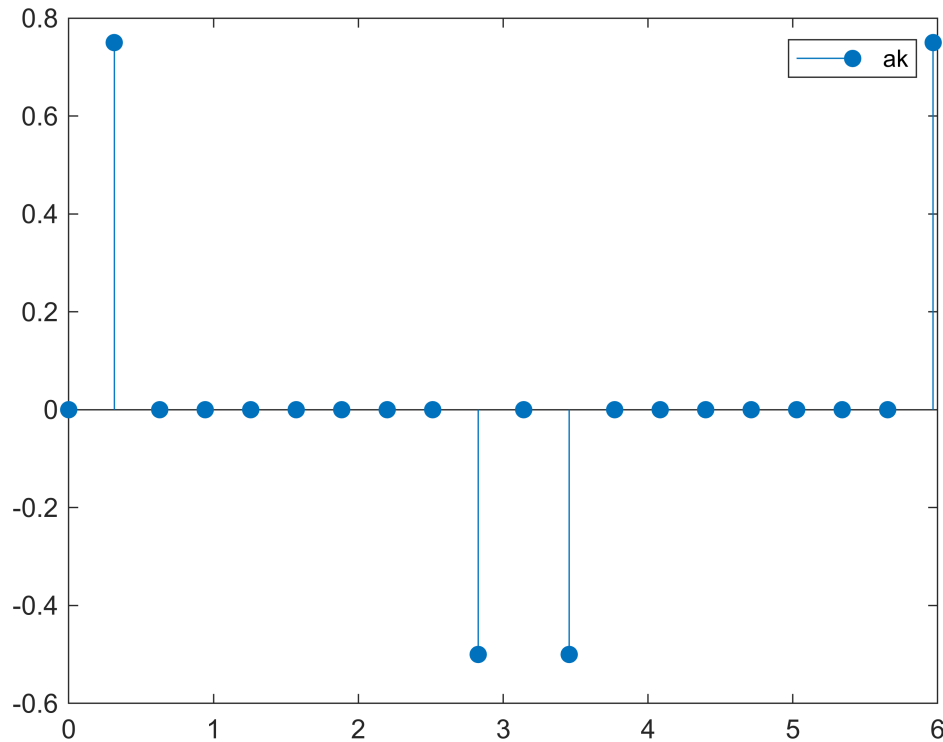
```
ak=[0 0.75 zeros(1,7) -0.5 0 -0.5 zeros(1,7) 0.75];
```

wk 可表示为

```
wk=0:pi/10:(pi/10*19);
```

以 wk 为横轴画出 ak 的图像

```
stem(wk,ak,'filled')
legend('ak');
```



结论：对于高通系统来说，当 k 为奇数时的频率被放大，偶数时缩小，对于低通系统来说，当 k 为偶数时的频率放大，奇数时缩小；x[n]作为输入信号时，对于系统 1， ω_1 与 ω_{19} 会被放大， ω_9 与 ω_{11} 会被减弱，对于系统 2， ω_1 与 ω_{19} 会被减弱， ω_9 与 ω_{11} 会被放大。

3.8d 定义 x_20 是 x[n] 在 0 至 19 范围内的向量集合，求 x[n] 在 -20 至 99 范围内的向量集合 x，画出 x[n] 的图像

分析：用 ifft 方法

用 ifft 函数求 x_20

```
x_20=(20).*ifft(ak)
```

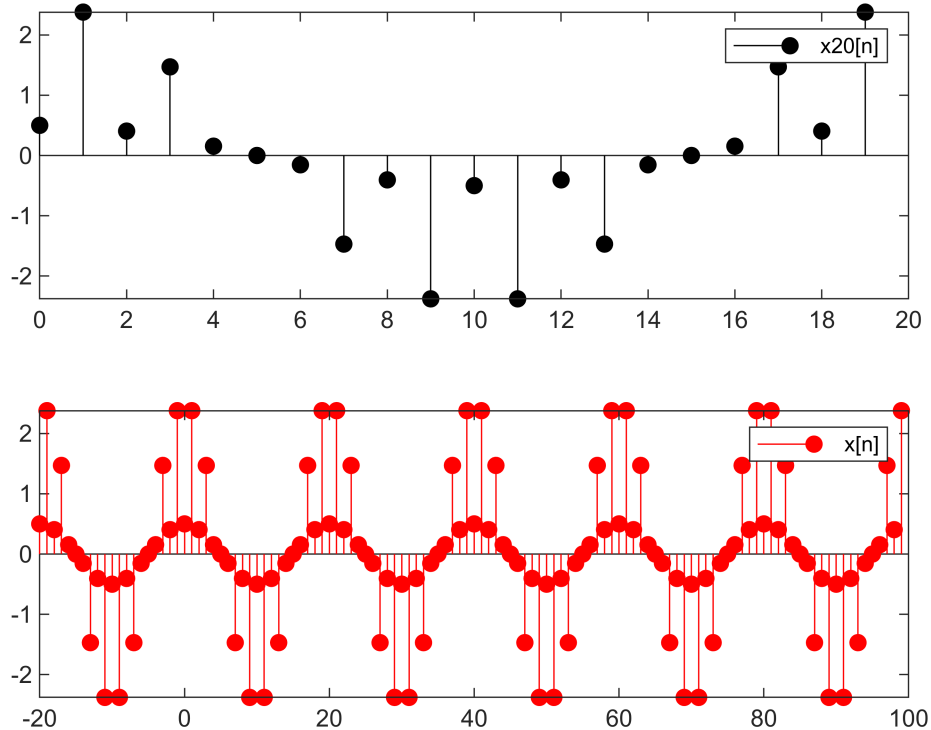
```
x_20 = 1x20
    0.5000    2.3776    0.4045    1.4695    0.1545   -0.0000   -0.1545   -1.4695 ...
```

x 可被表示为

```
xn=[x_20 x_20 x_20 x_20 x_20 x_20];
```

画出 x 的图像

```
subplot(2,1,1);
stem(0:19,x_20,'k','filled');
legend('x20[n]')
subplot(2,1,2);
stem(-20:99,xn,'r','filled');
legend('x[n]')
```



3.8e 用 `filter` 函数计算系统 1,2 的输出 y_1, y_2 ，画出图像，陈述哪个包含更多的高频能量

分析：画出图像，哪个更密集则频率更高

用 filter 函数计算 y_1, y_2

```
n=0:99;  
x=[x_20 x_20 x_20 x_20 x_20]
```

```
x = 1x100
    0.5000    2.3776    0.4045    1.4695    0.1545   -0.0000   -0.1545   -1.4695 ...
```

```
y1=filter(b1,a1,x)
```

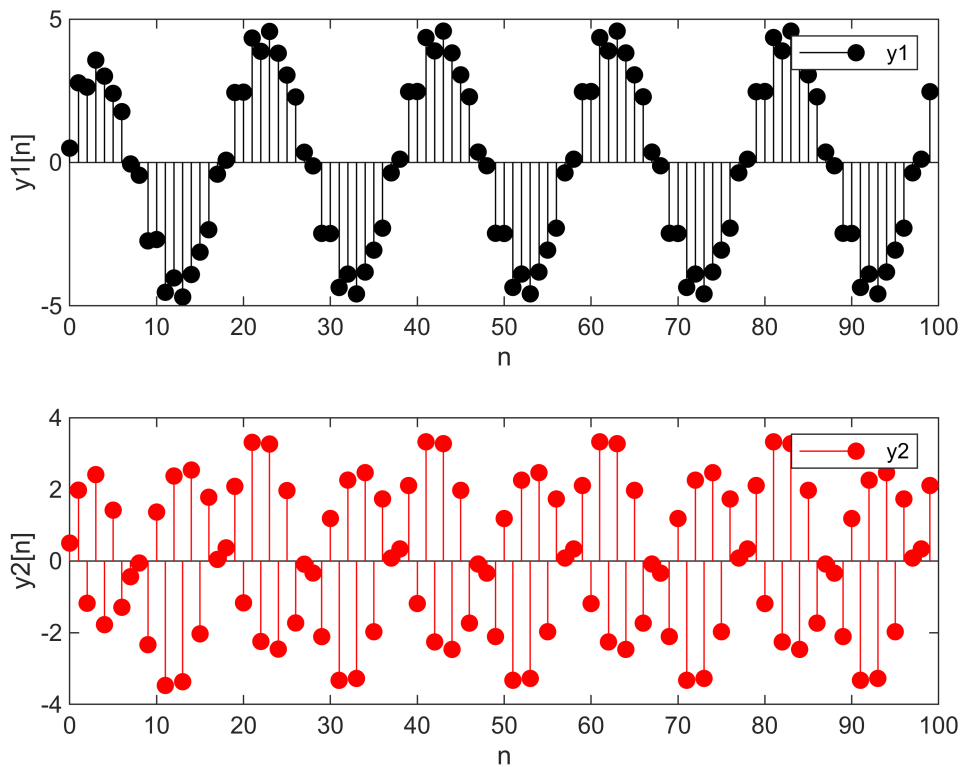
```
y1 = 1x100
0.5000    2.7776    2.6266    3.5708    3.0111    2.4089    1.7726    -0.0514 ...
```

```
y2=filter(b2,a2,x)
```

```
y2 = 1×100  
0.5000    1.9776   -1.1776    2.4115   -1.7747    1.4198   -1.2903   -0.4372 ...
```

画出 0-99 范围内的 y1,y2 图像

```
subplot(2,1,1);  
stem(0:99,y1,'k','filled');  
xlabel('n');  
ylabel('y1[n]');  
legend('y1')  
subplot(2,1,2);  
stem(0:99,y2,'r','filled');  
xlabel('n');  
ylabel('y2[n]');  
legend('y2')
```



结论：由图，y2 的频率更高,有更多高频能量

3.8f 用 fft 函数计算 0-19 范围内 y1,y2 的傅里叶级数并画图

分析：用 fft 方法

令 y1_20 表示 0-19 的 y1 集合

```
ky=0:pi/10:(pi/10*19);
```



```
y1_20=y1(1:20)
```

```
y1_20 = 1×20  
0.5000    2.7776    2.6266    3.5708    3.0111    2.4089    1.7726   -0.0514 ...
```

令 y2_20 表示 0-19 的 y2 集合

```
y2_20=y2(1:20)
```

```
y2_20 = 1×20  
0.5000    1.9776   -1.1776    2.4115   -1.7747    1.4198   -1.2903   -0.4372 ...
```

用 fft 函数计算 a_y1,a_y2

```
a_y1=1/20.*fft(y1_20)
```

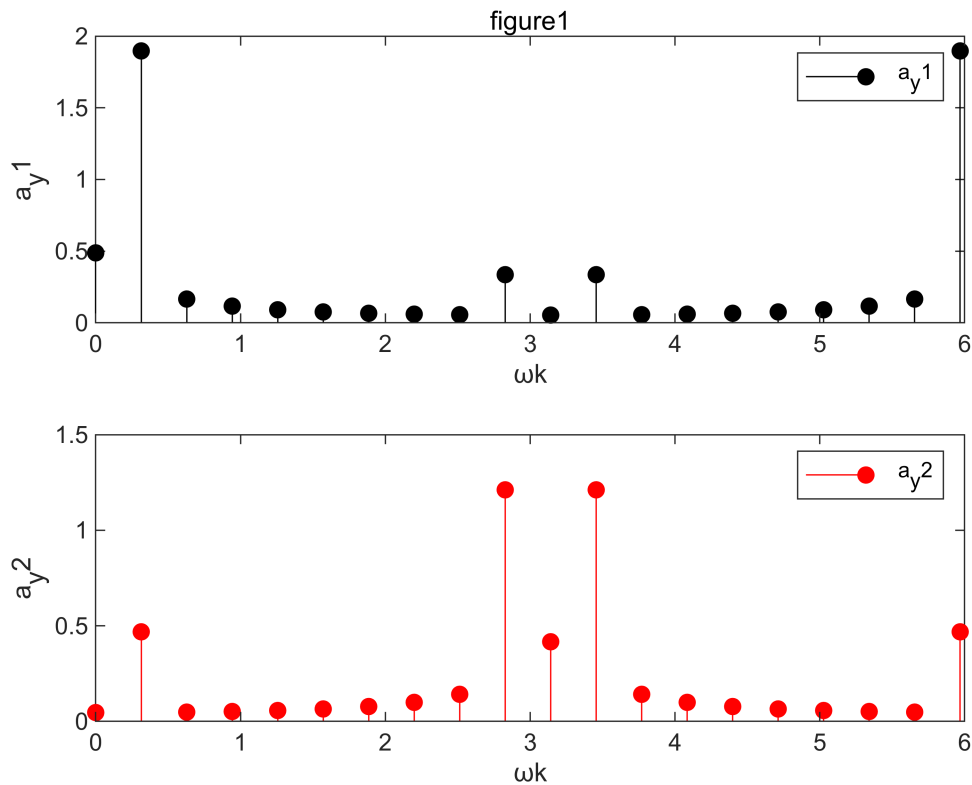
```
a_y1 = 1×20 complex  
-0.4881 + 0.0000i    1.3187 - 1.3632i   -0.0997 + 0.1328i   -0.0739 + 0.0903i ...
```

```
a_y2=1/20.*fft(y2_20)
```

```
a_y2 = 1×20 complex  
0.0463 + 0.0000i    0.4641 + 0.0652i    0.0468 + 0.0134i    0.0475 + 0.0209i ...
```

画出两者图像

```
figure;  
subplot(2,1,1);  
stem(ky,abs(a_y1),'k','filled');  
xlabel('wk');  
ylabel('a_y1');  
legend('a_y1');  
title('figure1');  
subplot(2,1,2);  
stem(ky,abs(a_y2),'r','filled');  
xlabel('wk');  
ylabel('a_y2');  
legend('a_y2');
```



结论：绘制的 y_1 和 y_2 的离散时间傅里叶级数系数图像如上所示，根据绘制的图像，其结论与(e)问中 y_1 含有更多的低频能量， y_2 含有更多的高频能量相符合。

3.9a. 题目要求使用 `lsim` 函数在 0-20 上作出频率响应 $H(s)$ 储存于 $y(t)$ 中，并在 $t=10:20$ 上作出 $x(t)$ 与 $y(t)$ 的图像。标出振幅与相位变化并做出解释。

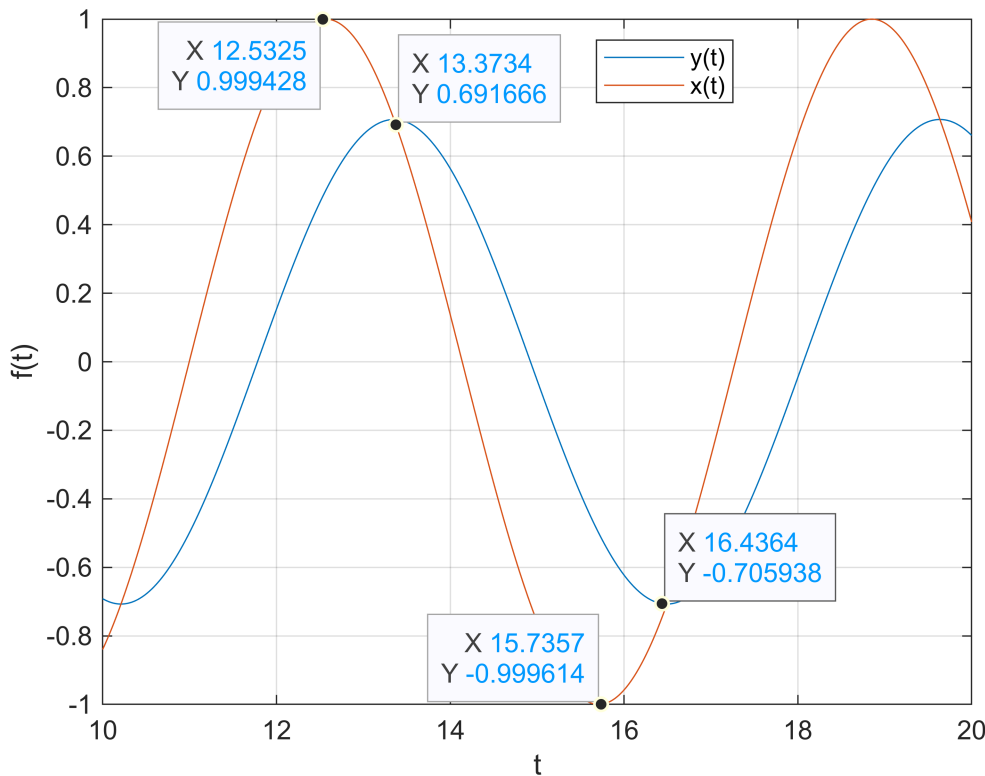
分析：利用 `lsim` 函数按照题目要求操作即可。

```
t = linspace(0,20,1000);
x = cos(t);
A = [1,1];
B = 1;
y = lsim(B,A,x,t);
figure
plot(t,y);
hold on;
plot(t,x);
xlim([10,20])
xlabel('t')
ylabel('f(t)')
grid on
legend('y(t)', 'x(t)');
hold off;
```

```

xlim([10.0 20.0])
legend("Position", [0.54411,0.80791,0.1743,0.11017])
ax = gca;
chart = ax.Children(1);
datatip(chart,13.37,0.6917);
datatip(chart,15.74,-0.9996,"Location","northwest");
datatip(chart,12.53,0.9994,"Location","southwest");
chart = ax.Children(2);
datatip(chart,16.44,-0.7059);

```



3.9b 题目要求作出当输入信号为 $x_2(t)$ 时系统的输出，同时作图。

分析：按照题目要求即可。

```

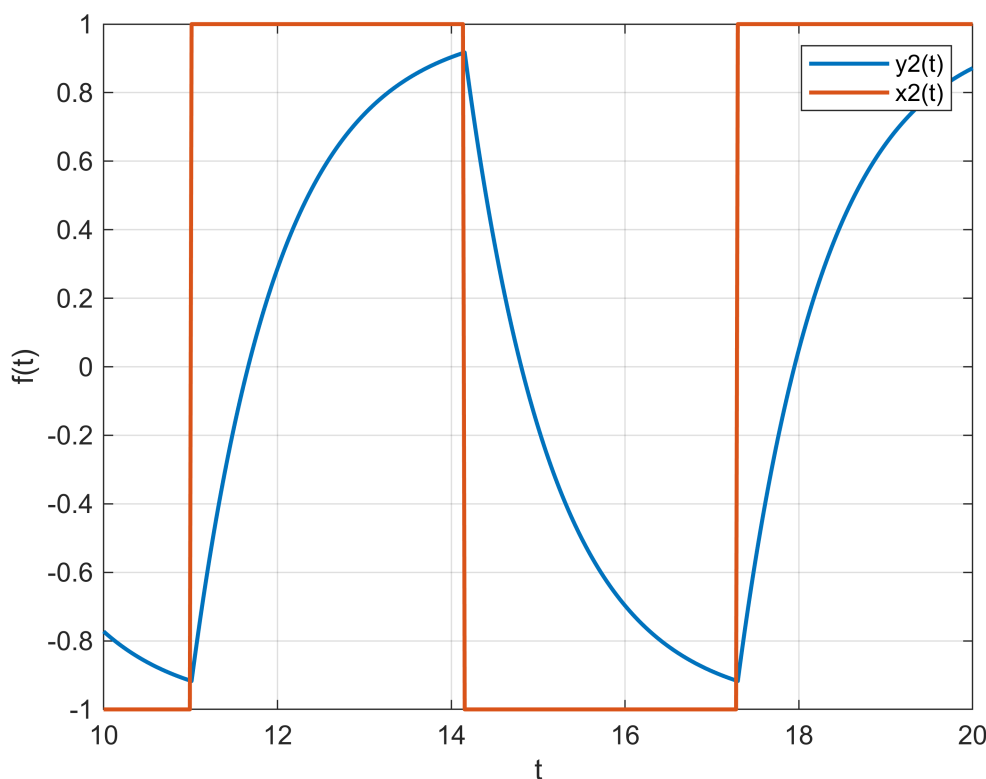
x2 = cos(t);
x2(x2>0)=ones(size(x2(x2>0)));
x2(x2<0)=-ones(size(x2(x2<0)));
y2=lsim(B,A,x2,t);
figure;
plot(t,y2,"LineWidth",1.5);
hold on;
plot(t,x2,"LineWidth",1.5);
xlim([10,20])

```

```

xlabel('t')
ylabel('f(t)')
grid on;
legend('y2(t)', 'x2(t)')
hold off;

```



结论：如上图所示。

3.9c 题目要求计算出 x_2 的傅里叶级数并利用 10 个频率从小到大傅里叶系数作出对应 5 个谐波分量，随后在图中作出 $x_2(t)$ 拟合图像和原图图像。

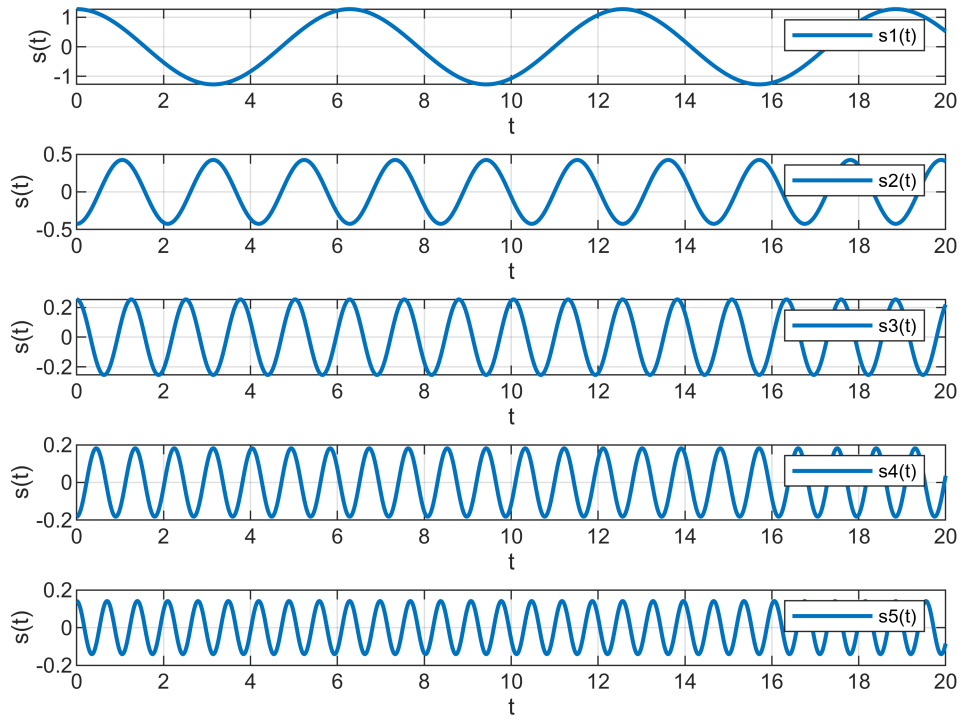
分析：由公式易得傅里叶级数表达式为 $a_k = 2 \sin(k\pi/2)/(k\pi)$ 。后根据公式作图

```

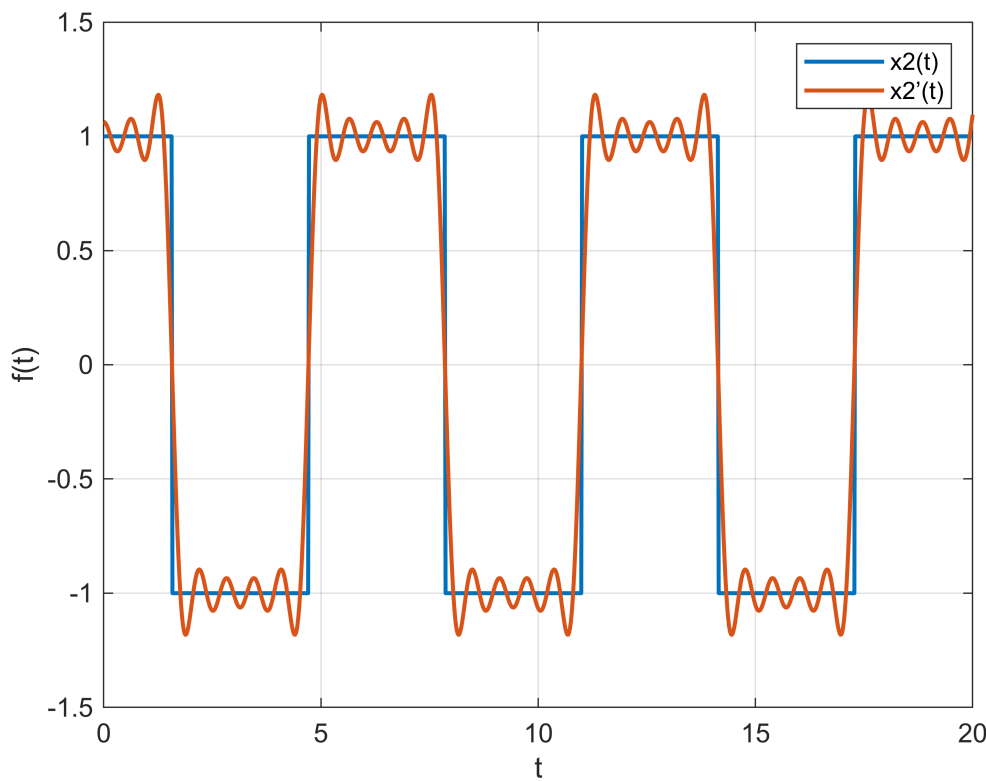
s1=2*sin(pi/2)/(pi)*exp(1i*t)+2*sin(-pi/2)/(-pi)*exp(-1i*t);
s2=2*sin(3*pi/2)/(3*pi)*exp(3*1i*t)+2*sin(-3*pi/2)/(-3*pi)*exp(-3*1i*t);
s3=2*sin(5*pi/2)/(5*pi)*exp(5*1i*t)+2*sin(-5*pi/2)/(-5*pi)*exp(-5*1i*t);
s4=2*sin(7*pi/2)/(7*pi)*exp(7*1i*t)+2*sin(-7*pi/2)/(-7*pi)*exp(-7*1i*t);
s5=2*sin(9*pi/2)/(9*pi)*exp(9*1i*t)+2*sin(-9*pi/2)/(-9*pi)*exp(-9*1i*t);
figure
subplot(5,1,1), plot(t,s1,'linewidth',1.5),xlabel('t'),ylabel('s(t)'),grid on,legend('s1(t)');
subplot(5,1,2), plot(t,s2,'linewidth',1.5),xlabel('t'),ylabel('s(t)'),grid on,legend('s2(t)');
subplot(5,1,3), plot(t,s3,'linewidth',1.5),xlabel('t'),ylabel('s(t)'),grid on,legend('s3(t)');
subplot(5,1,4), plot(t,s4,'linewidth',1.5),xlabel('t'),ylabel('s(t)'),grid on,legend('s4(t)');

```

```
subplot(5,1,5), plot(t,s5,'linewidth',1.5),xlabel('t'),ylabel('s(t)'),grid on,legend('s5(t)');
```



```
figure
plot(t,x2,t,s1+s2+s3+s4+s5,'linewidth',1.5),xlabel('t'),ylabel('f(t)'),grid on,legend('x2(t)',
```

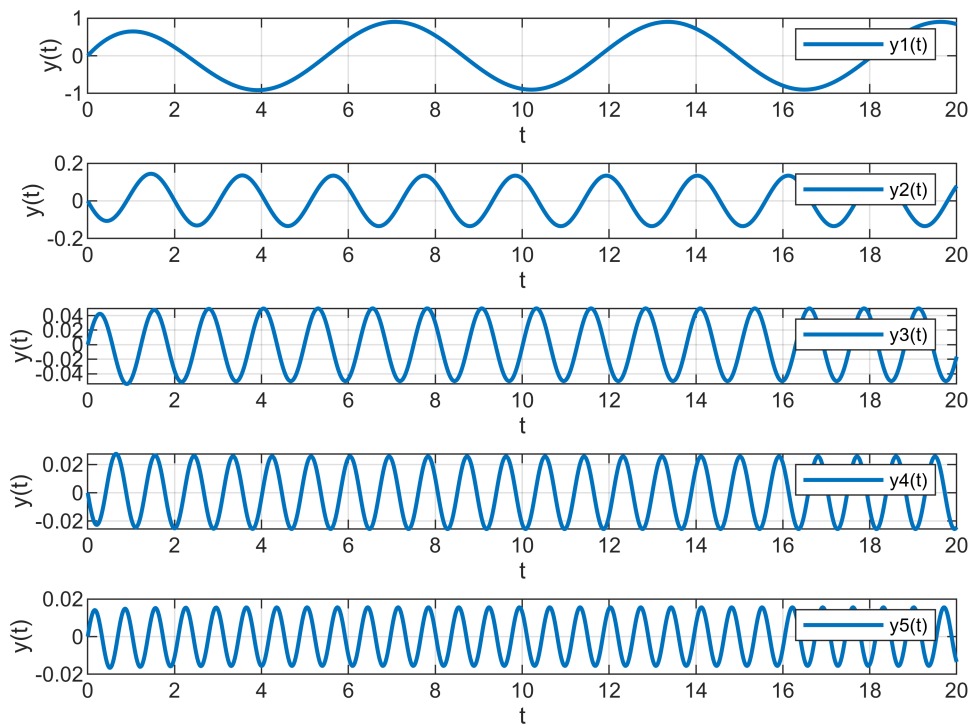


结论：作图如图所示。(注：本来想用循环写的，但是系统一直报错，因此改变代码方式)

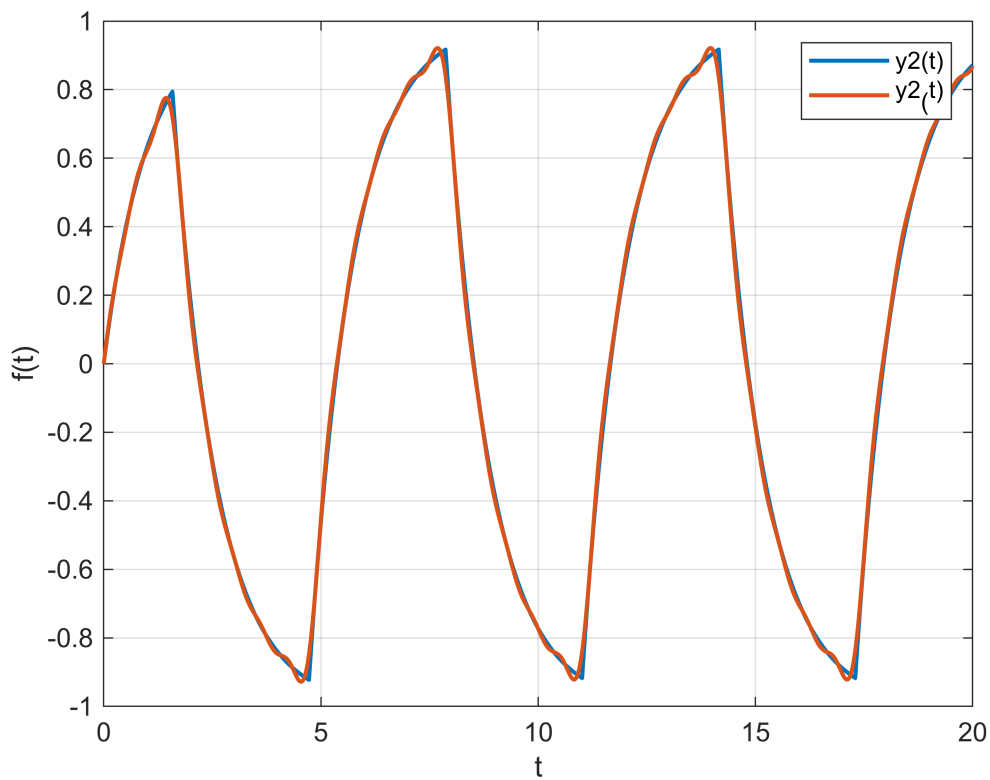
3.9d 题目要求利用系统的线性性质，分别算出 RC 电路对 S1-S5 的响应 y_1 - y_5 ，并作出五个拟合信号的和 y_{sum} 的响应并作图。

分析：按照题目要求操作即可

```
y1 = lsim(B,A,s1,t);
y2_1 = lsim(B,A,s2,t);
y3 = lsim(B,A,s3,t);
y4 = lsim(B,A,s4,t);
y5 = lsim(B,A,s5,t);
sum_of_s = s1+s2+s3+s4+s5;
y_sum=lsim(B,A,sum_of_s,t);
figure
subplot(5,1,1), plot(t,y1,'linewidth',1.5),xlabel('t'),ylabel('y(t)'),grid on,legend('y1(t)');
subplot(5,1,2), plot(t,y2_1,'linewidth',1.5),xlabel('t'),ylabel('y(t)'),grid on,legend('y2(t)');
subplot(5,1,3), plot(t,y3,'linewidth',1.5),xlabel('t'),ylabel('y(t)'),grid on,legend('y3(t)');
subplot(5,1,4), plot(t,y4,'linewidth',1.5),xlabel('t'),ylabel('y(t)'),grid on,legend('y4(t)');
subplot(5,1,5), plot(t,y5,'linewidth',1.5),xlabel('t'),ylabel('y(t)'),grid on,legend('y5(t)');
```



```
figure
plot(t,y2,t,y_sum,'linewidth',1.5),xlabel('t'),ylabel('f(t)'),grid on,legend('y2(t)','y2_(t)')
```



结论：图像如图所示。

3.9e 题目要求比较 RC 系统对方波 x_2 的响应和其对傅里叶级数系数和 $ssum$ 的响应，并解释相似性。

分析：两个响应如 (c) 问所示。对连续周期信号，傅里叶级数的综合公式将其拟合为谐波信号之和，当 k 趋于无穷大时，拟合函数与原函数近乎相同。题目中选取傅里叶级数的前十项，对 x_2 进行拟合，结果与 $x_2(t)$ 非常相似，因此两者作为输入进入同一个系统的响应也是几乎相同的。

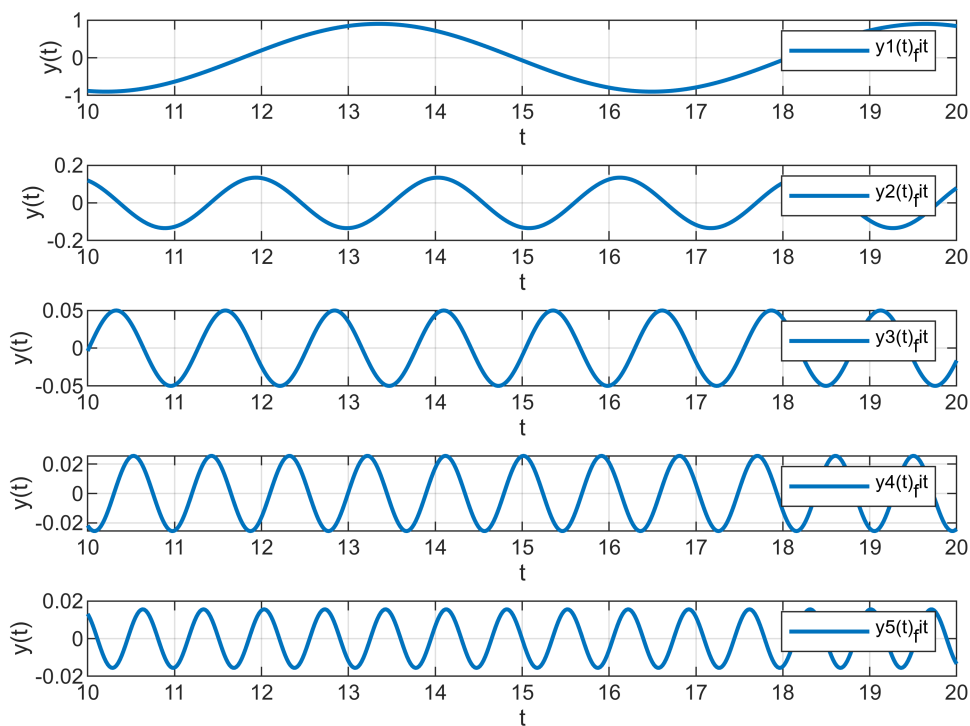
结论：CTFS 实现了对连续周期信号的拟合，因此输出也相同。

3.9 f 题目要求用傅里叶级数写出 y_1 - y_5 ，将其与前面运用频率响应得到的 y_1 - y_5 作比较来验证前面问题求得结果的正确性。

分析： $H(j\omega_k)=1/(1+jk)$ ，则其傅里叶级数 $a_k=1/(1+jk)*2*\sin(k\pi/2)/(k\pi)$ 。代入拟合出 y_k 即可。

```
y1_f = 1/(1+3*1i)*2*sin(pi/2)/(pi)*exp(1i*t)+1/(1-1i)*2*sin(-pi/2)/(-pi)*exp(-1i*t);
y2_f = 1/(1+3*1i)*2*sin(3*pi/2)/(3*pi)*exp(3*1i*t)+1/(1-3*1i)*2*sin(-3*pi/2)/(-3*pi)*exp(-3*1i*t);
y3_f = 1/(1+5*1i)*2*sin(5*pi/2)/(5*pi)*exp(5*1i*t)+1/(1-5*1i)*2*sin(-5*pi/2)/(-5*pi)*exp(-5*1i*t);
y4_f = 1/(1+7*1i)*2*sin(7*pi/2)/(7*pi)*exp(7*1i*t)+1/(1-7*1i)*2*sin(-7*pi/2)/(-7*pi)*exp(-7*1i*t);
y5_f = 1/(1+9*1i)*2*sin(9*pi/2)/(9*pi)*exp(9*1i*t)+1/(1-9*1i)*2*sin(-9*pi/2)/(-9*pi)*exp(-9*1i*t);

figure
subplot(5,1,1),plot(t,y1,'LineWidth',1.5),xlabel('t'),ylabel('y(t)'),grid on,legend('y1(t)_fit')
subplot(5,1,2),plot(t,y2,'LineWidth',1.5),xlabel('t'),ylabel('y(t)'),grid on,legend('y2(t)_fit')
subplot(5,1,3),plot(t,y3,'LineWidth',1.5),xlabel('t'),ylabel('y(t)'),grid on,legend('y3(t)_fit')
subplot(5,1,4),plot(t,y4,'LineWidth',1.5),xlabel('t'),ylabel('y(t)'),grid on,legend('y4(t)_fit')
subplot(5,1,5),plot(t,y5,'LineWidth',1.5),xlabel('t'),ylabel('y(t)'),grid on,legend('y5(t)_fit')
```

figure

```
subplot(5,1,1),plot(t,y1_f,'LineWidth',1.5,'Color',[87,142,98]/255),xlabel('t'),ylabel('y(t)').
```

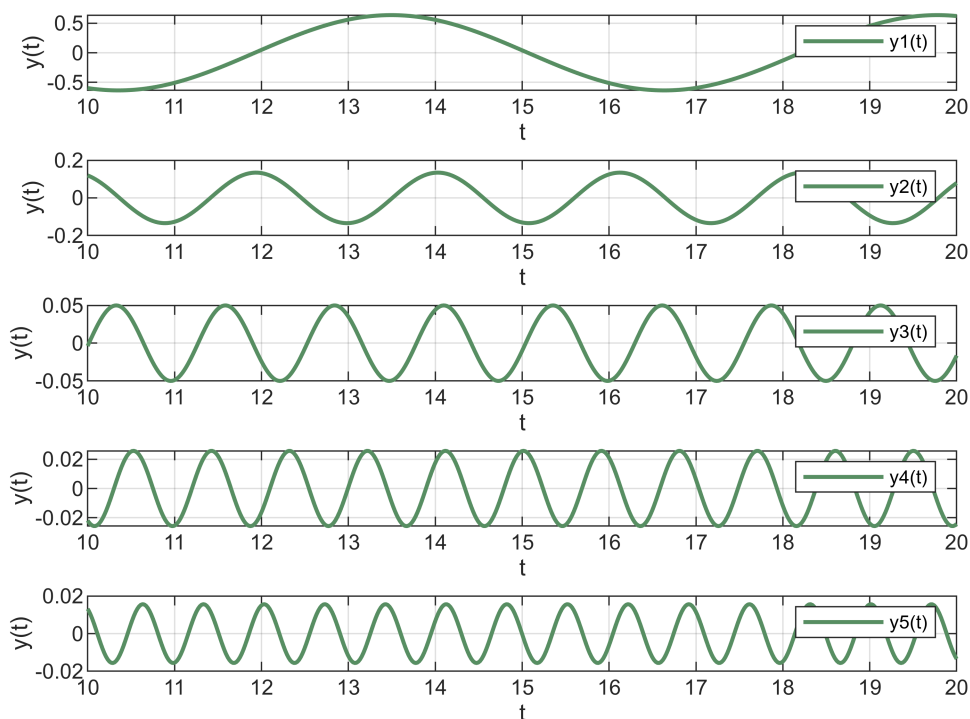
警告：复数 x 和/或 y 参数的虚部已忽略。

```
subplot(5,1,2),plot(t,y2_f,'LineWidth',1.5,'Color',[87,142,98]/255),xlabel('t'),ylabel('y(t)').
```

```
subplot(5,1,3),plot(t,y3_f,'LineWidth',1.5,'Color',[87,142,98]/255),xlabel('t'),ylabel('y(t)').
```

```
subplot(5,1,4),plot(t,y4_f,'LineWidth',1.5,'Color',[87,142,98]/255),xlabel('t'),ylabel('y(t)').
```

```
subplot(5,1,5),plot(t,y5_f,'LineWidth',1.5,'Color',[87,142,98]/255),xlabel('t'),ylabel('y(t)').
```



结论：

3.10a.题目要求得出求 DTFS 的过程中的总计算次数：

分析：由傅里叶级数计算公式得，计算每个 a_k 时，需要将 $x[N]$ 乘以 $e^{(-j\omega_k n)}$ 共 N 次，最后除以 N ，共 $N+1$ 次乘法运算。

随后将所有 N 个计算结果相加，共 $N-1$ 次运算，总共需要 $2N$ 次计算。一共有 N 个 a_k 需要计算。故总计算数 $2*N*N=2N^2$ 。

结论：求 DTFS 的总计算数为 $2*N^2$ 。

3.10b 题目要求计算出以 $x[n] = (0.9)^n$, $N=8,32,64,128,256$ 时自定义算法 DTFS 计算 FS 的计算量，并将结果写入 dtfscomps 数组中。

分析：flops 函数算法效率低且过失，这里使用 timeit 方法，写出 $x[n]$ 并计算。为提高精度，对每个 N 值计算 50 次并取平均值，单位为取毫秒。之后再以 $\log_2(N)$ 为横坐标， t 为纵坐标作图。

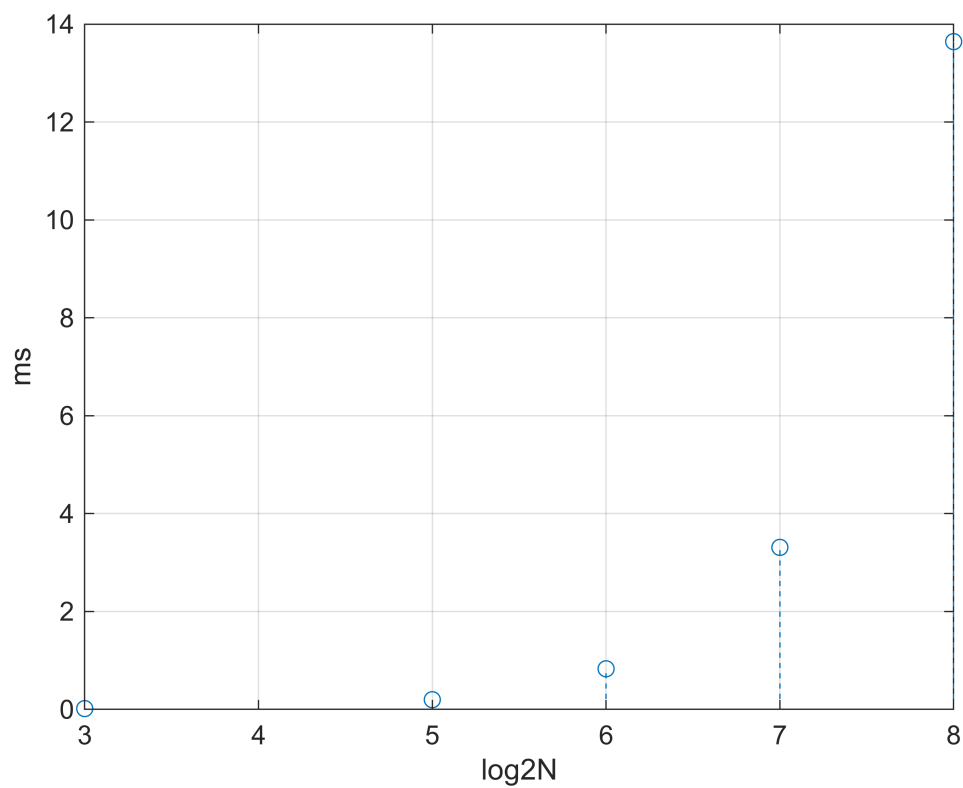
```
figure
N0=[8,32,64,128,256];
dtfscomps=zeros(1,5);
for i = 1:length(N0)
    N=N0(i);
```

```

x = 0.9.^(0:N-1);
t=0;
for j=0:49
    f = @()dtfs(x,0);
    t_timeit = timeit(f);
    t = t+t_timeit;
end
t = t/50;
dtfscomps(i)=t*1000;
end

N1 = [3,5,6,7,8];
stem(N1,dtfscomps,'LineStyle','--')
xlabel('log2N')
ylabel('ms')
grid on

```

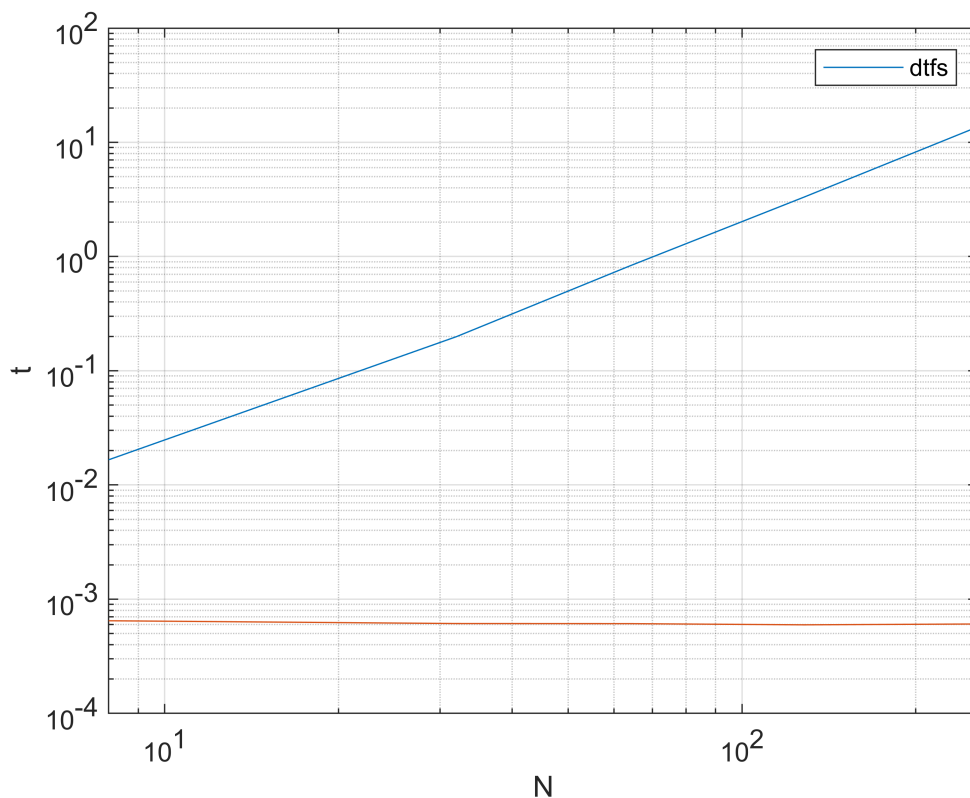


结论：运算速度较慢，由图可知 N 变大后计算时间变长，且基本满足式子 $2*N^2$ ，与上题中的分析吻合。

3.10c 题目要求求出 $N=8, 32, 64, 128, 256$ 时 fft 计算傅里叶级数的运算量，将结果保存在 fftcomps 数组中；然后使用 loglog 方法将 dtfscomps 和 fftcomps 对 N 画在同一张图上。

分析：同 b，使用 timeit 方法，写出 $x[n]$ 并计算。为提高精度，对每个 N 值计算 50 次并取平均值，单位为取毫秒。

```
figure
N0=[8,32,64,128,256];
fftcomps=zeros(1,5);
for i = 1:length(N0)
    N=N0(i);
    x = 0.9.^(0:N-1);
    t=0;
    for j=0:49
        f = @()fft(x,0);
        t_timeit = timeit(f);
        t = t+t_timeit;
    end
    t = t/50;
    fftcomps(i)=t;
end
fftcomps = fftcomps.*(1000);
loglog(N0,dtfscomps,N0,fftcomps)
grid on
xlabel('N')
ylabel('t')
legend('dtfs');
```



结论：由图可知，使用快速傅里叶变换后，计算量 N 大大减少，且随着 N 的增加，使用自定义方法的计算量增长很大，而使用 `fft` 方法计算量几乎无增加。

3.10d 已知 $h[n]$ 和 $x[n]$ 两个函数都是周期为 N 的信号。要求得出周期卷积函数的周期和计算量。

分析：因为 $y[n+N] = x[n] * h[n+N] = x[n] * h[n] = y[n]$ 因此 $y[n]$ 是周期为 N 的信号，计算卷积中的一项需要做 $N-1$ 次加法和 N 次乘法，因此计算一个周期后计算量为 $N * (N-1) + N * N = 2N^2 - N$ 。与 $O(N^2)$ 的计算量相符。

结论： $y[n]$ 是周期为 N 的信号，总运算次数为 $2N^2 - N$

3.10e 题目要求计算出 $N=40$ 时， $x[n]=0.9^n$ 和 $h[n]=0.5^n$ 两个函数的周期卷积的结果，并记录运行时间。

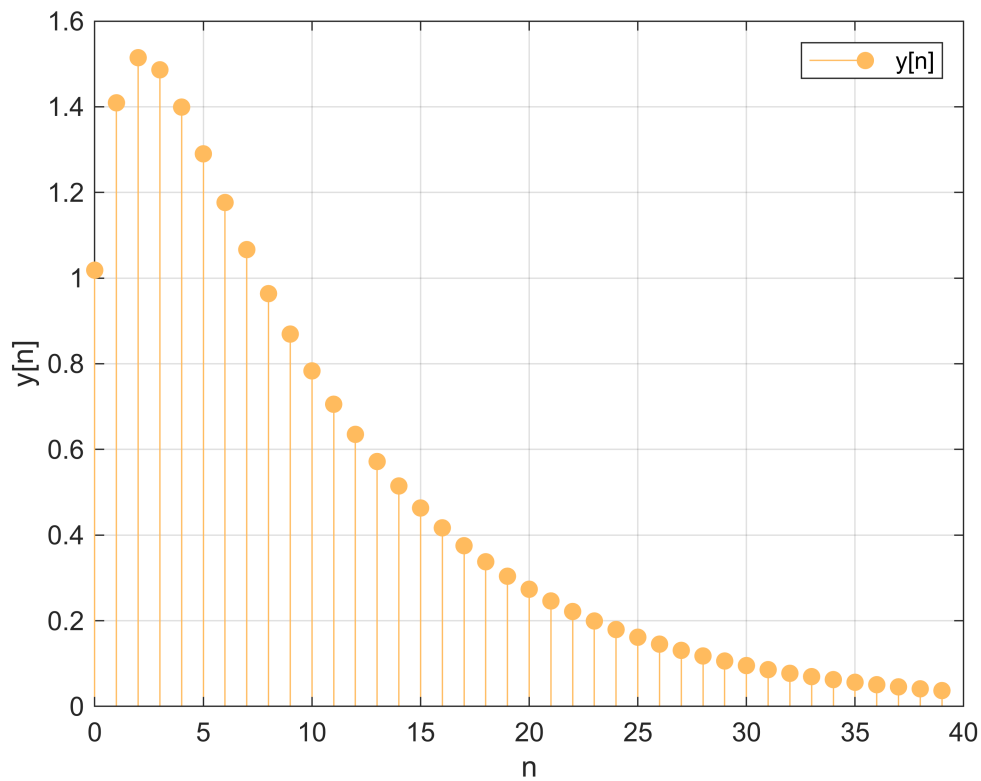
分析：按要求写代码即可。

```
figure
xn4 = 0:39;
xn = 0.9.^xn4;
hn = 0.5.^xn4;
zn = conv([xn xn],hn);
time = 0;
for i = 0:1:49
    f = @()(conv([xn xn],hn));
```

```

t_timeie2 = timeit(f);
time = time+t_timeie2;
end
yn=zn(41:80);
stem(xn4,yn,'filled','Color',[255,187,94]/255);
grid on
xlabel('n');
ylabel('y[n]')
legend('y[n]')

```



```
f40c = time/50
```

```
f40c = 2.5207e-06
```

结论：图像和时间如上所示。

3.10 f 题目要求同 e ,但是将周期改为了 $N=80$.

分析：同 e 问。

```

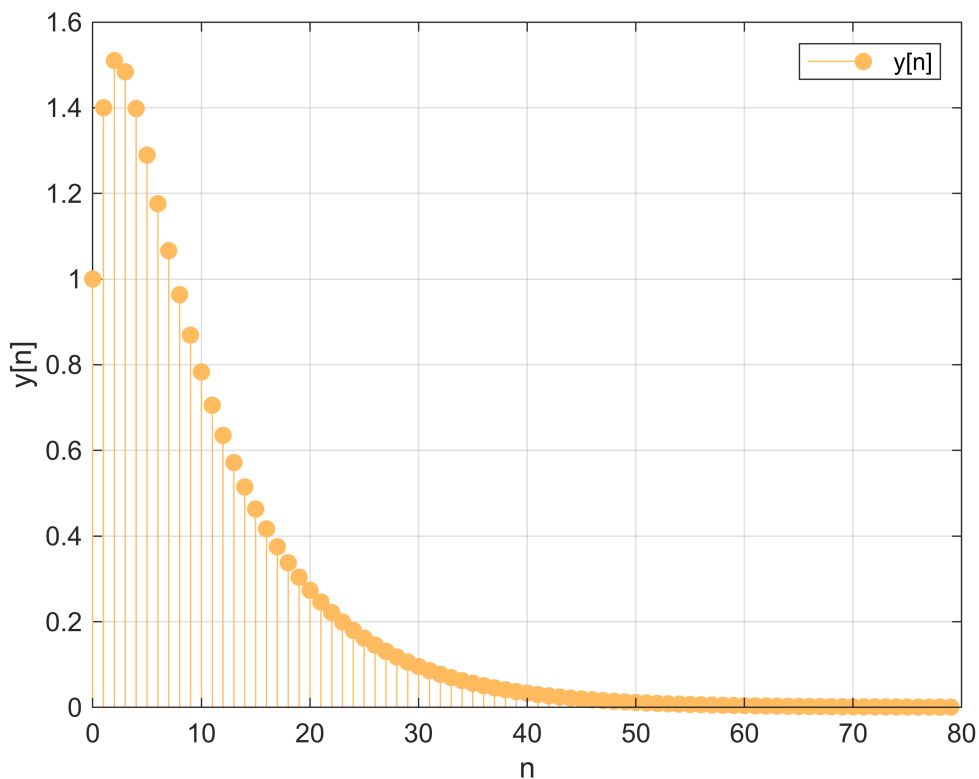
figure
xn4 = 0:79;
xn = 0.9.^xn4;

```

```

hn = 0.5.^xn4;
zn = conv([xn xn],hn);
time2 = 0;
for i = 0:1:49
    f = @()(conv([xn xn],hn));
    t_timeie2 = timeit(f);
    time2 = time2+t_timeie2;
end
yn=zn(81:160);
stem(xn4,yn, 'filled', 'Color',[255,187,94]/255);
grid on
xlabel('n');
ylabel('y[n]')
legend('y[n]')

```



```
f80c = time2/50
```

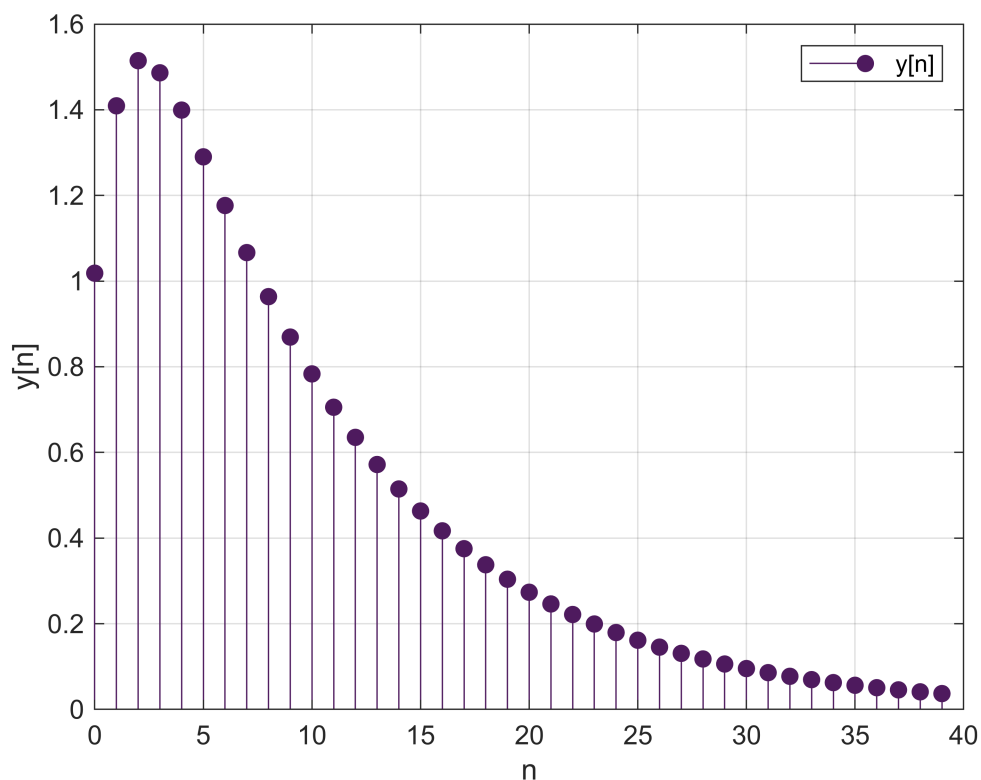
```
f80c = 3.1061e-06
```

结论：图像和时间如上所示。

3.10 g 题目要求计算出 $h[n]$, $x[n]$ 运用 FFT 计算出二者 DTFS, 随后通过 DTFS 的卷积系数计算出 $y[n]$ 的 DTFS, 再通过反变换 iff 还原出 $y[n]$, 作图与 e 比较, 并储存计算时间。

分析：按题目要求操作即可。

```
figure
xn5 = 0:39;
xn = 0.9.^xn5;
hn = 0.5.^xn5;
xnft = 1/40*fft(xn);
hnft = 1/40*fft(hn);
ynft = 40*xnft.*hnft;
time = 0;
for i = 0:1:49
    f = @(40*iFFT(ynft));
    t_timeie2 = timeit(f);
    time = time+t_timeie2;
end
yn=40*iFFT(ynft);
stem(xn5,yn,'filled','Color',[78,26,94]/255);
grid on
xlabel('n');
ylabel('y[n]');
legend('y[n]');
```




```
f40f = time
```

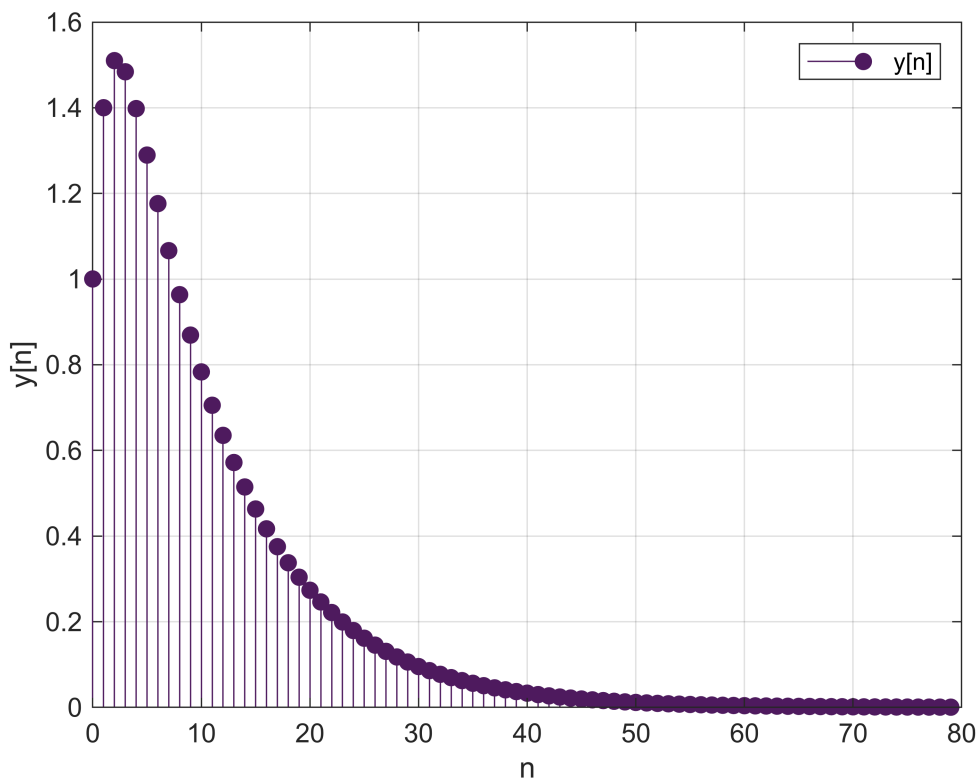
```
f40f = 4.1634e-05
```

结论：如图所示，图像和时间如上所示，图像与 e 问几乎相同

3.10 h 题目要求计算出 $h[n]$ ， $x[n]$ 在 $N=80$ 时运用 FFT 计算出二者 DTFS，随后通过 DTFS 的卷积系数计算出 $y[n]$ 的 DTFS，再通过反变换还原出 $y[n]$ ，作图与 f 比较，并储存计算时间。

分析：按照题目要求操作即可

```
figure
xn6 = 0:79;
xn = 0.9.^xn6;
hn = 0.5.^xn6;
xnft = 1/80*fft(xn);
hnft = 1/80*fft(hn);
ynft = 80*xnft.*hnft;
time = 0;
for i = 0:1:49
    f = @(80*ifft(ynft));
    t_timeie2 = timeit(f);
    time = time+t_timeie2;
end
yn=80*ifft(ynft);
stem(xn6,yn,'filled','Color',[78,26,94]/255);
grid on
xlabel('n');
ylabel('y[n]');
legend('y[n]');
```



```
f80f = time
```

```
f80f = 5.7484e-05
```

3.10 i 题目要求计算 f_{40c}/f_{40f} 和 f_{80c}/f_{80f} , 得出对于 $N=40$ 或 $N=80$ 哪种算法更高效。并预测当 $N>80$ 时哪种算法更好。

```
r1 = f40c/f40f
```

```
r1 = 0.0605
```

```
r2 = f80c/f80f
```

```
r2 = 0.0540
```

结论：两者的值均远小于 1, 由此得出直接对原信号进行卷积运算更好。